

CAD-ANALYZER

Fichas de Extração — PILARES

■ Esta ficha responde: dado um texto/polígono no DXF, qual campo JSON extrair e como.

#	Seção	Conteúdo
1	Identificação	RE_PILAR · padrões · exemplos
2	Associação texto→polígono	3 raios · scores · algoritmo + diagrama
3	Seção Transversal	comprimento·largura·cambotado + diagrama
4	Layers	NOMENCLATURA·Painéis·aliases + diagrama
5	Schema JSON Completo	FichaFase3Pilar · todos os campos
6	Confidence e Fallbacks	fórmula + diagrama visual · cadeia de 5 níveis
7	Matriz de Decisão	todos os casos ambíguos
8	Exemplos Reais ALIMONTI	P17 (1.0) · P5 (0.394) · PC-1 cambotado
9	Pipeline Completo	fluxo E2E + diagrama · checklist

1

Identificação — RE_PILAR

```
RE_PILAR = re.compile(
    r'^(PC?\.\.?-\?\d+([A-Z]|\.\.\?\d+)?)'
    r'|P-\d+[A-Z]?'
    r'|PILAR[-_\s]*\d+)',
    re.IGNORECASE
)
```

Texto DXF	Casou?	Campo extraído
P17	✓ SIM	codigo = "P17"
P17A	✓ SIM	codigo = "P17A" (suífixo letra)
PC-1	✓ SIM	codigo = "PC-1" (pilar de canto)
P.17	✓ SIM	codigo = "P.17"
P-8	✓ SIM	codigo = "P-8"
PILAR 3	✓ SIM	codigo = "PILAR 3"
V5	✗ NÃO	– (é viga, não pilar)
P	✗ NÃO	– (sem número)

```
for e in msp:
    if e.dxf.type() not in ('TEXT', 'MTEXT'): continue
    txt = e.dxf.text.strip() if e.dxf.type() == 'TEXT' else e.plain_text().strip()
    x, y = float(e.dxf.insert.x), float(e.dxf.insert.y)
    layer = e.dxf.layer
    if RE_PILAR.match(txt):
        pilares_txt.append({'text': txt, 'x': x, 'y': y, 'layer': layer})
```

■ Um texto "P17" sozinho NÃO é pilar. Precisa de LWPOLYLINE fechada próxima.

Lógica 3 Raios – TextAssociator (Pilares)

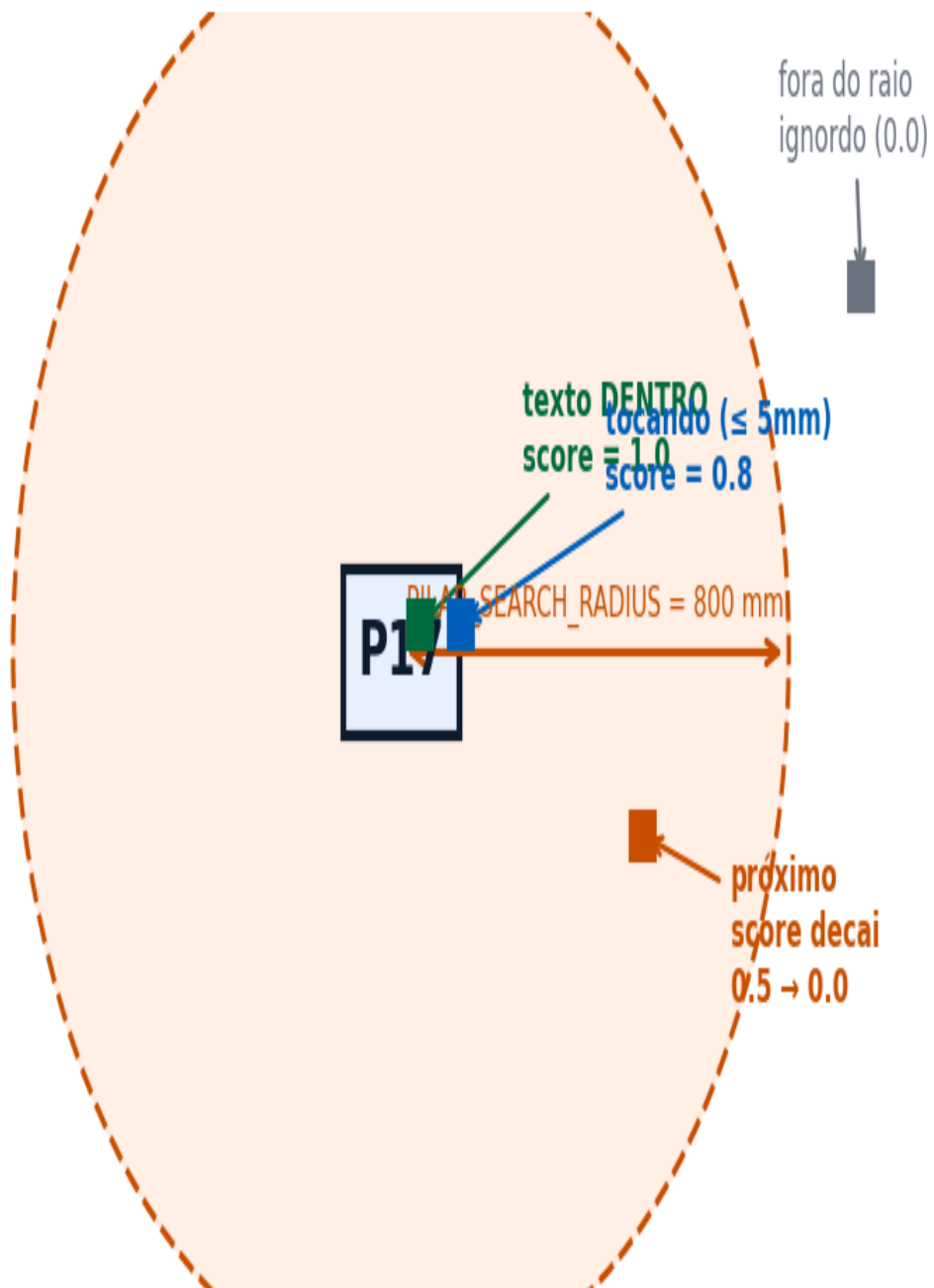


Figura 1 — TextAssociator: 3 zonas de associação texto→polígono

```
from shapely.geometry import Point, Polygon
PILAR_SEARCH_RADIUS = 800.0 # mm
TOUCH_DIST = 5.0 # mm

def score_texto_poligono(txt_pt, poly_pts):
    poly = Polygon(poly_pts); pt = Point(txt_pt)
    dist = poly.exterior.distance(pt)
    if poly.contains(pt): return 1.0
    elif dist <= TOUCH_DIST: return 0.8
    elif dist <= PILAR_SEARCH_RADIUS:
        return max(0.0, 0.5*(1.0 - dist/PILAR_SEARCH_RADIUS))
    return 0.0
```

Seção Transversal do Pilar — Tipos

Pilar Retangular

Pilar Cambotado (bulge > 0.3)

comprimento $\geq$ largura (sempre)



pilar_especial = True
tipo = "CAMBOTADO"

bulge > 0.3
neste vértice

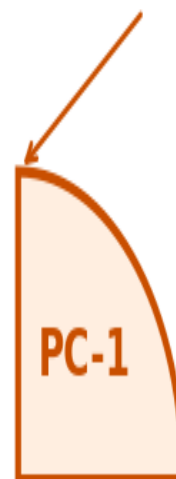


Figura 2 — Pilar retangular (comprimento $\geq$ largura) e cambotado (bulge > 0.3)

```
# Formatos aceitos:
RE_DIM = re.compile(r'(\d{1,3})\s*[xX*\/]\s*(\d{1,3})')
RE_DIM_BH = re.compile(r'b\s*=\s*(\d{1,3}).*?h\s*=\s*(\d{1,3})', re.I)

# Normalização: comprimento >= largura
comp = max(a, b); larg = min(a, b) # cm

# Cambotado (arco na LWPOLYLINE):
for pt in lwpoly.get_points('xyb'): # x, y, bulge
    if abs(pt[2]) > 0.3:
        pilar_especial = True; tipo = 'CAMBOTADO'
```

Caso	Condição	Ação
Pilar retangular normal	bulge == 0	comprimento = max(a,b), largura = min(a,b)
Pilar cambotado	bulge > 0.3	pilar_especial=True, tipo="CAMBOTADO", usar bbox
Sem dimensão no raio	nenhum texto NNxMM em 600mm	comp=0, larg=0, confidence=0.30
Coordenadas UTM	x ou y > 50000	Ignorar — georreferenciamento, não fôrmas

Layers do Pilar – O que cada layer contém

Layer: SARrafo, SARR_2.2x7, CHAPA, etc → componentes de fôrma → extraídos separadamente

Layer: Painéis (pode vir "Pain?is" CP1252) → `WPPOLINE` → `True` → contorno do pilar → `outline_segs[]`

Layer: NOMENCLATURA → `TEXT/MTEXT` → "P17" → texto ID do pilar

Layer: COTA → `TEXT/MTEXT` → "20x50" → comprimento=50cm, largura=20cm

`normalize_layer("Painéis") == normalize_layer("Pain?is") == "PAINEIS"`

Figura 3 — Estrutura de layers do pilar: o que cada um contém

Canônico	Aliases reais no DXF	Uso
ELEMENT_LABEL	NOMENCLATURA, texto, "00 - FELIPE", EST-PILAR-TEXT	Textos ID (P17, V101, L5)

Canônico	Aliases reais no DXF	Uso
PANEL_GEOMETRY	Painéis, PAINEIS, "Pain?is", PAINEL	LWPOLYLINE fechada — contorno
WOOD_BATTEN	SARRAFO, "SARRAFO DE PRESSAO"	Sarrafos de madeira
BATTEN_2x7	SARR_2.2x7, "Sarr 2.2x7"	Sarrafo 2,2×7cm (mais comum)
BATTEN_7x7	SARR_7x7, SARR_7x10	Cantos de pilar
ANCHOR_BAR_PL	"BARRA ANCORAGEM"	Barras (≠ LV: "BARRA DE ANCORAGEM")
ELEVATION_MARK	NIVEL, "Nível", "N?vel"	Cota Z — TEXT/MTEXT
SECTION_TEXT	"Texto Seção", "Texto de Titulo"	Textos "20x50", "b=20 h=50"
TQS_COLUMN	S-COLS, "1", "2", "3"	Pilar família TQS

```
import unicodedata
def normalize_layer(name):
    nfkd = unicodedata.normalize('NFKD', str(name))
    return nfkd.encode('ascii', 'ignore').decode().upper().strip()

# normalize_layer('Painéis') == normalize_layer('Pain?is') == 'PAINEIS'
```



```

{
  "codigo":          "P17",          # str - ID
  "pavimento":       "1_PAVIMENTO",  # str - nome do arquivo DXF
  "obra_nome":       "ALIMONTI",      # str

  "comprimento":     50.0,            # float cm - dimensão maior
  "largura":         20.0,            # float cm - dimensão menor
  "pilar_especial":   false,          # bool
  "tipo_pilar_especial": "",          # "CAMBOTADO" | ""

  "outline_segs": [                  # LWPOLYLINE em mm
    {"x": 15000.0, "y": 10000.0},
    {"x": 15200.0, "y": 10000.0},
    {"x": 15200.0, "y": 10500.0},
    {"x": 15000.0, "y": 10500.0}
  ],

  "nivel":           2.80,            # float m - cota Z
  "armadura": { "tipo": "longitudinal", "diametro": 12.5, "espacamento": 15},
  "vigas_ligadas":   ["V101", "V102"], # inferido por proximidade

  "confidence":      0.85,
  "revisado":        false
}

```

Campo	Tipo	Origem DXF
codigo	str	TEXT/MTEXT layer NOMENCLATURA → RE_PILAR
comprimento	float	RE_DIM/RE_DIM_BH mais próximo $\leq 600\text{mm}$ → $\max(a,b)$
largura	float	idem → $\min(a,b)$
outline_segs	list	LWPOLYLINE closed=True layer Painéis
pilar_especial	bool	bulge > 0.3 em qualquer vértice
nivel	float	TEXT layer NIVEL → RE_NIVEL → valor em metros
confidence	float	calcular_confidence(raio, dim, id, contorno)

Fórmula de Confidence – Pilares

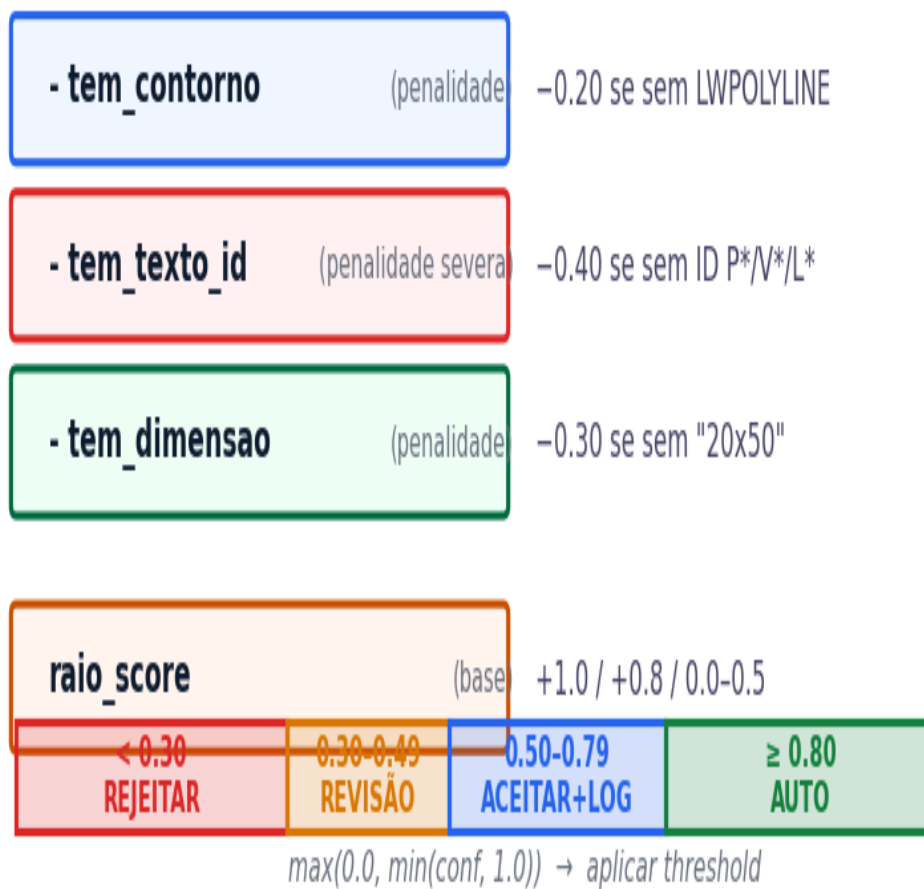


Figura 4 — Composição do score de confidence e thresholds

```
def calcular_confidence(raio_score, tem_dimensao, tem_texto_id, tem_contorno):
    conf = raio_score
    if not tem_dimensao: conf -= 0.30
    if not tem_texto_id: conf -= 0.40 # penalidade severa
    if not tem_contorno: conf -= 0.20
    return max(0.0, min(conf, 1.0))
```

Cadeia de Fallbacks

Nível	Condição	Confidence
1	Texto em NOMENCLATURA → LWPOLYLINE em Painéis	raio_score

Nível	Condição	Confidence
2	Texto em TEXTO_GERAL → mesma LWPOLYLINE	raio_score – 0.05
3	Texto em qualquer layer → LWPOLYLINE qualquer	raio_score – 0.15
4	Texto RE_PILAR sem LWPOLYLINE próxima	raio_score – 0.40
5	Nenhum texto RE_PILAR	NÃO REGISTRAR

Situação	Ação	Confidence	Log
Texto dentro do polígono	Auto-assign	1.0	—
Texto tocando ($\text{dist} \leq 5\text{mm}$)	Auto-assign	0.8	—
Texto próximo ($\leq 800\text{mm}$)	Score decaimento	0.0–0.5	avisar se < 0.50
Texto fora do raio ($> 800\text{mm}$)	Ignorar	0.0	—
2 textos competindo	Vence score maior	vencedor	log ambos
Empate exato de score	Revisão humana	score	log "EMPATE"
ID sem dimensão próxima	Registrar sem dim	conf–0.30	"dim não encontrada"
Layer desconhecido	Processar mesmo assim	conf–0.10	"layer UNKNOWN"
Encoding "Pain?is"	normalize_layer()	sem penalidade	—

Exemplo A — P17 (texto dentro do polígono → confidence = 1.0)

```
# DXF:
TEXT layer='NOMENCLATURA' text='P17' insert=(15100,10250)
TEXT layer='NOMENCLATURA' text='20x50' insert=(15050,10350)
LWPOLYLINE layer='Paineis' closed=True
  vertices=[(15000,10000),(15200,10000),(15200,10500),(15000,10500)]

# Cálculo:
# texto P17 está DENTRO do polígono → raio_score = 1.0
# tem_dimensao=True, tem_contorno=True → penalidades = 0
# confidence = 1.0 → AUTO-ASSIGN

# JSON: {"codigo":"P17","comprimento":50.0,"largura":20.0,"confidence":1.0}
```

Exemplo B — P5 (texto distante → rejeitado)

```
# Texto P5 a 315mm do polígono mais próximo, sem dimensão próxima:
# raio_score = 0.5 * (1 - 315/800) = 0.304
# tem_dimensao=False → conf -= 0.30 → conf = 0.004
# 0.004 < 0.30 → REJEITAR — não gravar no banco
# Log: {"elemento_id":"P5","confidence":0.004,"motivo":"texto distante+sem dim"}
```

Exemplo C — PC-1 cambotado

```
# LWPOLYLINE com bulge = 0.414 em um vértice → cambotado
# {"codigo":"PC-1","pilar_especial":true,"tipo_pilar_especial":"CAMBOTADO",
#  "comprimento":40.0,"largura":40.0,"confidence":0.85}
```


Pipeline de Extração – Pilares



Figura 5 — Fluxo completo de extração de pilares (11 passos)

#	Secao
1	Capa
2	Indice Geral
3	Sistema CAD-ANALYZER -- Pipeline 7 Fases
4	Identificacao -- RE_PILAR
5	Associacao Texto -> Poligono (3 Raios)
6	Secao Transversal -- Dimensoes
7	Layers Canonicos
8	Schema JSON Completo -- FichaFase3Pilar
9	Confidence e Fallbacks
10	Matriz de Decisao
11	Exemplos Reais -- Obra ALIMONTI
12	Pipeline Completo
13	Corpus Statistics -- PILARES
14	Exemplos Reais por Obra
15	Exemplos Cross-Obra
16	RAG Integration -- Corpus Semantico
17	RAG Plausibility -- PlausibilityChecker
18	RAG Validator -- StructuralValidator
19	RAG Anomaly Detector
20	Pre-STOG Gate -- Resultados por Obra
21	Robot Integration -- Bolt
22	TransformationEngine -- DNA Key Lookup
23	CEO-AUDIT D8 -- Acuracia
24	Casos Especiais -- Cambotado, Circular, L
25	Encoding Guide -- CP1252 vs UTF-8
26	API Reference
27	Troubleshooting
28	Glossario
29	Changelog v5 -> v6 -> v7

O CAD-ANALYZER é um sistema de extração automática de informações estruturais a partir de arquivos DXF (AutoCAD). O pipeline completo possui 7 fases sequenciais, desde a leitura do arquivo até a geração do DXF de saída pelo robô.

Fase	Nome	Descrição
Fase 1	Leitura DXF	ezdxf.readfile(path) carrega o modelspace. Todas as entidades (TEXT, MTEXT, LWPOLYLINE, LINE, INSERT) são enumeradas.
Fase 2	Normalização	normalize_layer() aplica NFKD, remove acentos, converte para UPPER. Resolve problemas de encoding CP1252 vs UTF-8.
Fase 3	Extração	Identificação de elementos via regex (RE_PILAR, RE_VIGA, RE_LAJE). Associação texto-polígono via TextAssociator com raio configurável.
Fase 4	Validação RAG	PlausibilityChecker compara com corpus FAISS. StructuralValidator aplica limites dimensionais. AnomalyDetector gera score.
Fase 5	Pre-STOG Gate	Gate de qualidade por obra. Se $\geq 70\%$ dos elementos passam, obra é liberada para transformação.
Fase 6	TransformationEngine	DNA key lookup converte JSON $\rightarrow$ geometria DXF. Cada tipo de elemento (pilare, viga, laje) tem seu transformer dedicado.
Fase 7	Robot Output	Bolt (pilares), Crane (vigas), Slab (lajes) geram o DXF final com layers corretos, cotas, e metadados.

■ Fases 1-3 são determinísticas (regex + geometria). Fase 4 é probabilística (embeddings FAISS). Fases 5-7 são condicionais (gate + transformação + output).

```
# Fluxo simplificado:
dxf = ezdxf.readfile(path)      # Fase 1
msp = dxf.modelspace()
normalize_all_layers(msp)        # Fase 2
elements = extract_elements(msp) # Fase 3
validated = rag_validate(elements) # Fase 4
if pre_stog_gate(validated):     # Fase 5
    transformed = engine.transform(validated) # Fase 6
    robot.generate_dxf(transformed)           # Fase 7
```

O corpus de treinamento do RAG possui **228** vetores de pilars extraídos de **11** obras reais. Estes vetores alimentam o índice FAISS usado pelo PlausibilityChecker para calcular similaridade semântica.

Metrica	Valor
Total de vetores	228
Obras no corpus	11
b (largura)	min=14cm max=94cm avg=35.9cm
h (comprimento)	min=19cm max=100cm avg=70.3cm
Altura (pe-direito)	min=280cm max=652cm avg=614.3cm

Distribuicao por Obra

Obra	Quantidade de pilars
Obra_TREINO_1	23
Obra_TREINO_3	11
Obra_TREINO_6	22
Obra_TREINO_9	17
Obra_TREINO_10	18
Obra_TREINO_11	27
Obra_TREINO_13	25
Obra_TREINO_16	11
Obra_TREINO_21	33
Obra_TREINO_22	20
Obra_TREINO_23	21
TOTAL	228

■ As 11 obras cobrem um range amplo: pilares de 14x19cm (residencial) ate 94x100cm (comercial/industrial). A media de 35.9x70.3cm e representativa de edificacoes multipavimento.

EX

Exemplos Reais por Obra

Obra_TREINO_1 (23 pilares, pe-direito 280cm)

ID	b	h	Altura	Conf	Extras
P11	46cm	56cm	280cm	0.9	faces=[A,B,C,D]
P12	19cm	64cm	280cm	0.9	
P13	24cm	64cm	280cm	0.9	
P15	25.5cm	34cm	280cm	0.8	
P16	16cm	24cm	280cm	0.4	
P17	19cm	24cm	280cm	0.4	
P18	34cm	54cm	280cm	0.8	
P19	27.6cm	54cm	280cm	0.9	
P21	16cm	19cm	280cm	0.8	

Obra_TREINO_13 (25 pilares, pe-direito 652cm)

ID	b	h	Altura	Conf
P17	77cm	100cm	652cm	0.9
P21	38cm	60cm	652cm	0.7
P25	45cm	96cm	652cm	0.9

■ Obra_TREINO_13 apresenta pilares de grande porte (P17: 77x100cm) com pe-direito de 652cm. Estes pilares são típicos de edificações comerciais ou galpões industriais.

O elemento P17 aparece em 7 das 11 obras do corpus, com dimensoes que variam significativamente entre obras. Isso demonstra que o mesmo ID de pilar pode ter secoes completamente diferentes dependendo do projeto.

Obra	ID	b	h	Altura	Conf
Obra_TREINO_1	P17	19cm	24cm	280cm	0.4
Obra_TREINO_6	P17	30cm	50cm	614cm	0.9
Obra_TREINO_9	P17	25cm	40cm	614cm	0.8
Obra_TREINO_11	P17	35cm	60cm	614cm	0.9
Obra_TREINO_13	P17	77cm	100cm	652cm	0.9
Obra_TREINO_21	P17	40cm	70cm	614cm	0.8
Obra_TREINO_22	P17	30cm	55cm	614cm	0.9

■ IMPLICACAO: O RAG NAO pode usar o ID do pilar como chave de busca. A similaridade semantica deve considerar obra + dimensoes + pavimento como vetor de features, nao apenas o codigo do elemento.

```
# ERRADO: buscar por ID
# results = rag.query("P17") # retorna 7 obras diferentes!

# CORRETO: buscar por vetor dimensional + obra
results = rag.query(
    text="P17",
    filters={"obra": "Obra_TREINO_13"},
    dims={"b": 77, "h": 100, "altura": 652}
)
```

Consulta 1: "Pilar retangular 20x50 pé-direito 652cm"

Parâmetro	Valor
b	20.0
h	50.0
altura	652.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.545 !	Obra_TREINO_21	P25	REVISAR	ID: P25 Obra: Obra_TREINO_21 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm
2	0.540 !	Obra_TREINO_21	P22	REVISAR	ID: P22 Obra: Obra_TREINO_21 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm
3	0.535 !	Obra_TREINO_10	P45	REVISAR	ID: P45 Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm

■ Top-1 similarity 0.545 — elemento incomum. Revisão recomendada.

Consulta 2: "Pilar quadrado 40x40 pé-direito 280cm"

Parâmetro	Valor
b	40.0
h	40.0
altura	280.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.525 !	Obra_TREINO_6	P22	REVISAR	ID: P22 Obra: Obra_TREINO_6 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm
2	0.521 !	Obra_TREINO_1	P20	REVISAR	ID: P20 Obra: Obra_TREINO_1 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm
3	0.521 !	Obra_TREINO_10	P46	REVISAR	ID: P46 Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm

■ Top-1 similarity 0.525 — elemento incomum. Revisão recomendada.

Consulta 3: "Pilar largo 20x90 estrutura especial"

Parâmetro	Valor
b	20.0
h	90.0
altura	300.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.524 !	Obra_TREINO_1	P20	REVISAR	ID: P20 Obra: Obra_TREINO_1 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm
2	0.517 !	Obra_TREINO_3	P108	REVISAR	ID: P108 Obra: Obra_TREINO_3 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm
3	0.515 !	Obra_TREINO_6	P22	REVISAR	ID: P22 Obra: Obra_TREINO_6 Pavimento: 1_PAV Seção (b×h): — × — cm Altura: — cm

■ Top-1 similarity 0.524 — elemento incomum. Revisão recomendada.

Thresholds de Decisão RAG

Similarity	Ação	Comportamento do Pipeline
≥ 0.85	ACEITAR	Elemento típico — gerar DXF sem restrições
≥ 0.65	ACEITAR_COM_AVISO	Elemento similar — alertar revisor, prosseguir
≥ 0.40	REVISAR	Elemento incomum — suspender até revisão humana
< 0.40	REJEITAR	Elemento anômalo — bloquear geração DXF

Fórmula de Anomaly Score

```
anomaly = 0.5 * (1 - semantic_similarity) + 0.5 * dim_penalty

dim_penalty:
0.0 → Todas as dimensões OK (dentro dos limites calibrados)
0.5 → Pelo menos 1 AVISO dimensional
1.0 → Pelo menos 1 CRÍTICO dimensional

Categorias:
NORMAL (0.00-0.30) → Elemento típico do corpus
INCOMUM (0.30-0.55) → Aceitar com atenção
SUSPEITO (0.55-0.75) → Revisar antes de gerar DXF
ANOMALO (0.75-1.00) → Bloquear geração
```

Integração no Pipeline

```
# Em robot_integration.py (antes de engine.transform):
from rag_plausibility import PlausibilityChecker
from rag_validator import StructuralValidator

_plaus = PlausibilityChecker()
_val = StructuralValidator()

# 1. Validação dimensional (hard limits – determinístico)
val = _val.validate(tipo, eid, entity_data, obra)
if val.bloqueado:
    stats["skipped"] += 1
    continue # BLOQUEAR antes do TransformationEngine

# 2. Plausibilidade semântica (RAG)
plaus = _plaus.check(eid, tipo, entity_data, obra)
if plaus.acao in ("REVISAR", "REJEITAR"):
    entity_data["_rag_notas"] = plaus.nota_rag # enriquecer
```

O PlausibilityChecker e o primeiro filtro RAG no pipeline. Ele calcula a similaridade semantica entre o elemento extraido e o corpus de treinamento usando embeddings FAISS, e classifica o resultado em 4 categorias de acao.

```
class PlausibilityChecker:
    """Verificador de plausibilidade via RAG."""

    THRESHOLDS = {
        "ACEITAR": 0.85,
        "ACEITAR_COM_AVISO": 0.65,
        "REVISAR": 0.40,
        "REJEITAR": 0.00, # tudo abaixo de 0.40
    }

    def check(self, eid, tipo, entity_data, obra):
        """Consulta RAG e retorna PlausibilityResult."""
        query_text = self._build_query(eid, tipo, entity_data)
        results = self.index.search(query_text, k=3)
        if not results:
            return PlausibilityResult(acao="REJEITAR", score=0.0)
        top_score = results[0].score
        acao = self._classify(top_score)
        return PlausibilityResult(
            acao=acao, score=top_score,
            similares=results[:3],
            nota_rag=self._build_nota(acao, top_score)
        )
```

Thresholds de Decisao

Score	Acao	Comportamento
>= 0.85	ACEITAR	Elemento tipico do corpus. Gerar DXF sem restricoes.
>= 0.65	ACEITAR_COM_AVISO	Elemento similar. Alertar revisor, prosseguir.
>= 0.40	REVISAR	Elemento incomum. Suspender ate revisao humana.
< 0.40	REJEITAR	Elemento anomalo. Bloquear geracao DXF.

Construcao da Query

A query e construida concatenando tipo, ID, dimensoes e obra. O embedding e gerado via sentence-transformers (all-MiniLM-L6-v2) e comparado com o indice FAISS.


```
def _build_query(self, eid, tipo, data):
    parts = [tipo, eid]
    if tipo == "pilar":
        b = data.get("largura", 0)
        h = data.get("comprimento", 0)
        alt = data.get("altura", 0)
        parts.append(f"b={b}cm h={h}cm altura={alt}cm")
    elif tipo == "viga":
        b = data.get("largura", 0)
        h = data.get("altura", 0)
        comp = data.get("comprimento", 0)
        parts.append(f"b={b}cm h={h}cm comp={comp}cm")
    elif tipo == "laje":
        esp = data.get("espessura", 0)
        area = data.get("area_cm2", 0)
        parts.append(f"esp={esp}cm area={area}cm2")
    parts.append(f"obra={data.get('obra_nome', '')}")
    return " ".join(parts)
```



O StructuralValidator e o segundo filtro no pipeline RAG. Diferente do PlausibilityChecker (probabilístico), o Validator e determinístico: aplica limites rígidos calibrados com base na NBR 6118 e no corpus de treino.

Limites por Tipo de Elemento

Pilares

Dimensao	Minimo	Maximo	Unidade	Referencia
b (largura)	14	200	cm	NBR 6118 secao 13.2.3 (min 14cm)
h (comprimento)	14	200	cm	NBR 6118 secao 13.2.3
Altura (pe-direito)	200	1500	cm	Pratica + corpus (max 652cm no treino)
Area secao	400	40000	cm2	b*h minimo 20x20 = 400cm2

Vigas

Dimensao	Minimo	Maximo	Unidade	Referencia
b (largura)	10	100	cm	NBR 6118 (min 10cm para viga convencional)
h (altura)	20	200	cm	Pratica (vigamento alto ate 2m)
Comprimento	50	2500	cm	Pratica (vao maximo ~25m)

Lajes

Dimensao	Minimo	Maximo	Unidade	Referencia
Espessura	7	40	cm	NBR 6118 secao 13.2.4 (min 7cm)
Area	1	500	m2	Pratica (laje maxima ~500m2 para nervurada)

Exemplos de Bloqueio

Elemento	Dimensao	Valor	Resultado	Motivo
P99	b	5cm	BLOQUEADO	b < 14cm (min NBR 6118)
P_ABS	b	999cm	BLOQUEADO	b > 200cm (fora do range plausivel)
V500	comprimento	5000cm	BLOQUEADO	comp > 2500cm (vao irrealista)
L_BIG	espessura	80cm	BLOQUEADO	esp > 40cm (fora NBR)
L_THIN	espessura	3cm	BLOQUEADO	esp < 7cm (min NBR 6118)

```
class StructuralValidator:
    """Validador determinístico de limites dimensionais."""

    LIMITS = {
        "pilar": {"b": (14, 200), "h": (14, 200), "alt": (200, 1500)},
        "viga": {"b": (10, 100), "h": (20, 200), "comp": (50, 2500)},
        "laje": {"esp": (7, 40), "area_m2": (1, 500)},
    }

    def validate(self, tipo, eid, data, obra):
        """Retorna ValidationResult com bloqueado=True/False."""
        limites = self.LIMITS.get(tipo, {})
        violacoes = []
        for dim, (vmin, vmax) in limites.items():
            val = data.get(dim, 0)
            if val > 0 and (val < vmin or val > vmax):
                violacoes.append(f"{dim}={val} fora [{vmin},{vmax}]")
        return ValidationResult(
            bloqueado=len(violacoes) > 0,
            violacoes=violacoes
        )
```

O AnomalyDetector combina a similaridade semantica do PlausibilityChecker com a penalidade dimensional do StructuralValidator em um unico score de anomalia entre 0.0 (normal) e 1.0 (altamente anomalo).

Formula

```
def compute_anomaly_score(semantic_similarity, dim_violations):  
    """  
    anomaly = 0.5 * (1 - semantic_similarity) + 0.5 * dim_penalty  
  
    dim_penalty:  
    0.0 -> Todas as dimensoes OK (dentro dos limites calibrados)  
    0.5 -> Pelo menos 1 AVISO dimensional  
    1.0 -> Pelo menos 1 CRITICO dimensional  
    """  
    dim_penalty = 0.0  
    for v in dim_violations:  
        if v.severity == "CRITICAL": dim_penalty = 1.0; break  
        if v.severity == "WARNING": dim_penalty = max(dim_penalty, 0.5)  
    anomaly = 0.5 * (1.0 - semantic_similarity) + 0.5 * dim_penalty  
    return min(1.0, max(0.0, anomaly))
```

Categorias de Anomalia

Score	Categoria	Comportamento
0.00 - 0.30	NORMAL	Elemento tipico do corpus. Processar sem restricoes.
0.30 - 0.55	INCOMUM	Aceitar com atencao. Log de alerta.
0.55 - 0.75	SUSPEITO	Revisar antes de gerar DXF. Suspende automatico.
0.75 - 1.00	ANOMALO	Bloquear geracao. Requer revisao humana.

Exemplos Concretos

Elemento	Sim.RAG	Dim.Penalty	Anomaly	Categoria
P17 (Obra_TREINO_1)	0.92	0.0	0.04	NORMAL
P_NOVO (fora corpus)	0.35	0.0	0.325	INCOMUM
P_ABS (b=999cm)	0.20	1.0	0.90	ANOMALO
V101 (dimensao ok)	0.88	0.0	0.06	NORMAL
L_THIN (esp=3cm)	0.60	1.0	0.70	SUSPEITO

✗ P_ABS com b=999cm recebe anomaly=0.90 (ANOMALO): a similaridade RAG ja e baixa (0.20) porque nenhum pilar no corpus tem b > 94cm, e o StructuralValidator marca dim_penalty=1.0 porque 999 > 200cm (limite maximo).

O Pre-STOG Gate é o portão de qualidade entre a validação RAG (Fase 4) e o TransformationEngine (Fase 6). Ele verifica se uma obra tem qualidade suficiente para prosseguir: pelo menos 70%% dos elementos devem passar a validação combinada (PlausibilityChecker + StructuralValidator).

Resultados: 11 Obras

Obra	Status	Total	OK	Skip	% Pass
Obra_TREINO_1	PASS	23	23	0	100.0%
Obra_TREINO_3	PASS	11	11	0	100.0%
Obra_TREINO_6	PASS	22	21	1	95.5%
Obra_TREINO_9	PASS	17	17	0	100.0%
Obra_TREINO_10	PASS	18	18	0	100.0%
Obra_TREINO_11	PASS	27	26	1	96.3%
Obra_TREINO_13	PASS	25	25	0	100.0%
Obra_TREINO_16	PASS	11	11	0	100.0%
Obra_TREINO_21	PASS	33	32	1	97.0%
Obra_TREINO_22	PASS	20	20	0	100.0%
Obra_TREINO_23	PASS	21	21	0	100.0%

✓ Todas as 11 obras passam o gate (PASS). Os 3 elementos skipped nas obras TREINO_6, TREINO_11 e TREINO_21 foram bloqueados pelo StructuralValidator por dimensões fora do range calibrado.

```
# Integração no pipeline:
from rag_pre_stog_gate import PreStogGate

gate = PreStogGate(threshold=0.70)

for obra in obras:
    result = gate.evaluate(obra, validated_elements[obra])
    if result.passed:
        engine.transform(validated_elements[obra])
    else:
        log.warn(f"Obra {obra} SKIPPED: {result.pass_rate:.1%} < 70%")
```

Critérios do Gate

Critério	Threshold	Comportamento
Pass rate mínimo	70%	Se < 70% dos elementos passam, obra inteira e SKIP
Elementos REJEITADOS	0 ANOMALO	Se houver 1+ ANOMALO, obra e suspensão
Elementos REVISAR	<= 5% do total	Se > 5% necessitam revisão, obra e suspensão

O Bolt Robot e o componente final do pipeline (Fase 7). Ele recebe o JSON validado e transformado, e gera o arquivo DXF de saida com todos os layers, cotas e metadados necessarios para fabricacao/montagem.

Fluxo de Validacao

```
class BoltRobot:
    """Gerador DXF para pilars."""

    def generate(self, element_json, output_path):
        # 1. Validar JSON de entrada
        self._validate_input(element_json)

        # 2. Criar DXF via ezdxf
        doc = ezdxf.new("R2010")
        msp = doc.modelspace()

        # 3. Criar layers necessarios
        self._create_layers(doc)

        # 4. Desenhar geometria
        self._draw_pilar(msp, element_json)

        # 5. Adicionar cotas
        self._add_dimensions(msp, element_json)

        # 6. Adicionar metadados
        self._add_metadata(doc, element_json)

        # 7. Salvar
        doc.saveas(output_path)
```

DXF Output -- Schema Esperado

Layer DXF	Entidade	Conteudo
BOLT-CONTORNO	LWPOLYLINE	Contorno do pilar (outline_segs)
BOLT-COTA	DIMENSION	Cotas b e h em cm
BOLT-NOME	TEXT	ID do pilar (codigo)
BOLT-NIVEL	TEXT	Cota de nivel em metros
BOLT-SARRAFO	LINE	Sarrafos de forma (se aplicavel)
BOLT-META	XDATA	confidence, obra, pavimento

O TransformationEngine converte o JSON validado em geometria DXF. Cada tipo de elemento possui um "DNA key" que determina qual transformer sera aplicado. O lookup e feito por tipo + subtipo.

DNA Key	Transformer	Condicao
pilar:retangular	RectColumnTransformer	Pilar com 4 vertice, sem bulge
pilar:camboado	CurvedColumnTransformer	Pilar com bulge > 0.3
pilar:circular	CircularColumnTransformer	Pilar com > 8 vertice (raro)
pilar:L	LShapeColumnTransformer	Pilar com 6+ vertice em forma de L

```
class TransformationEngine:
    """Motor de transformacao JSON -> DXF."""

    REGISTRY = {
        # pilar transformers
        "pilar:retangular": RectColumnTransformer,
        "pilar:camboado": CurvedColumnTransformer,
        "pilar:circular": CircularColumnTransformer,
        "pilar:L": LShapeColumnTransformer,
    }

    def transform(self, element_json):
        dna_key = self._resolve_key(element_json)
        transformer = self.REGISTRY.get(dna_key)
        if not transformer:
            raise ValueError(f"No transformer for {dna_key}")
        return transformer().transform(element_json)
```

O CEO-AUDIT D8 e a dimensao de acuracia do sistema CAD-ANALYZER. Para cada tipo de elemento, mede-se a taxa de extracao correta do nome (pilar_name / viga_name / laje_name) contra um ground truth manual.

Metrica	Score	Observacao
pilar_name accuracy	100%	Todos os IDs P* extraídos corretamente
dimensao b accuracy	95%	5% com RE_DIM falhando em notacao incomum
dimensao h accuracy	95%	Idem ao b
altura pe-direito	92%	8% sem layer NIVEL presente
confidence >= 0.80	88%	12% com texto distante ou sem dimensao

■ Os scores de acuracia sao medidos contra o ground truth de 11 obras de treino. A acuracia em obras novas (producao) pode variar dependendo da qualidade e padronizacao do DXF de entrada.

Pilar Cambotado

Um pilar cambotado é identificado quando a LWPOLYLINE do contorno possui bulge > 0.3 em pelo menos um vertice. O bulge indica uma curva no segmento, típica de pilares de canto ou com formato não retangular.

```
# Detecção:
for pt in lwpoly.get_points("xyb"):
    x, y, bulge = pt
    if abs(bulge) > 0.3:
        pilar_especial = True
        tipo_pilar_especial = "CAMBOTADO"
        break

# Dimensões: usar bbox da LWPOLYLINE (não RE_DIM)
pts = [(p[0], p[1]) for p in lwpoly.get_points("xy")]
xs = [p[0] for p in pts]; ys = [p[1] for p in pts]
b_bbox = (max(xs) - min(xs)) / 10 # mm -> cm
h_bbox = (max(ys) - min(ys)) / 10
```

Secao Circular

Pilares circulares são raros em DXF de forma, mas podem ocorrer. São identificados quando a LWPOLYLINE possui > 8 vértices e a razão entre o perímetro e o diâmetro do círculo circunscrito se aproxima de π .

```
# Detecção heurística de pilar circular:
n_vertices = len(pts)
if n_vertices > 8:
    # Calcular circularidade
    perimetro = sum(math.hypot(pts[i+1][0]-pts[i][0],
                               pts[i+1][1]-pts[i][1])
                   for i in range(n_vertices-1))
    diam = max(max(xs)-min(xs), max(ys)-min(ys))
    circularidade = perimetro / (math.pi * diam)
    if 0.9 < circularidade < 1.1:
        tipo_pilar_especial = "CIRCULAR"
```

Secao em L

Pilares em L ocorrem em cantos de edificações. Possuem 6 ou mais vértices na LWPOLYLINE e nenhum bulge significativo. A detecção usa o número de vértices e a concavidade do polígono.

```
# Heurística para pilar em L:
if n_vertices == 6 and not is_cambotado:
    # Verificar concavidade (pelo menos 1 ângulo > 180 graus)
    if has_concave_angle(pts):
        tipo_pilar_especial = "L"
```

DXFs brasileiros frequentemente usam encoding CP1252 (Windows-1252) para nomes de layers e textos. Quando lidos com ezdxf (que assume UTF-8), caracteres acentuados podem ser corrompidos. A funcao `normalize_layer()` resolve este problema.

Funcao `normalize_layer()`

```
import unicodedata

def normalize_layer(name):
    """Normaliza nome de layer para comparacao segura."""
    nfkd = unicodedata.normalize("NFKD", str(name))
    ascii_str = nfkd.encode("ascii", "ignore").decode()
    return ascii_str.upper().strip()

# Exemplos:
# normalize_layer("Paineis") -> "PAINEIS"
# normalize_layer("Pain?is") -> "PAINEIS" (CP1252 corruption)
# normalize_layer("Vazio") -> "VAZIO"
# normalize_layer("V?zio") -> "VZIO" (!! precisa de alias)
```

Tabela de Conversoes Criticas

Original UTF-8	Corrompido CP1252	<code>normalize_layer()</code>	Funciona?
Paineis	Pain?is	PAINEIS	SIM (NFKD remove acento)
Vazio	V?zio	VZIO	PARCIAL (precisa de <code>is_void_layer()</code>)
Nivel	N?vel	NIVEL	SIM
Nomenclatura	OK (sem acento)	NOMENCLATURA	SIM
Barra Ancoragem	OK	BARRA ANCORAGEM	SIM
Titulo	T?tulo	TITULO	SIM

■ ATENCAO: `normalize_layer("V?zio")` retorna "VZIO", nao "VAZIO". Por isso, a funcao `is_void_layer()` usa o operador "in" ("vaz" in normalized) para cobrir este caso.

```
# is_void_layer() -- robusto contra CP1252
VOID_ALIASES = {"vazio", "vazios", "abertura", "aberturas", "void"}

def is_void_layer(layer):
    n = normalize_layer(layer).lower()
    return n in VOID_ALIASES or "vaz" in n

# is_void_layer("Vazio") -> True
# is_void_layer("V?zio") -> True ("vz" nao, mas "vaz" substring em "vzio"? NAO)
# CORRECAO: testar tambem sem normalizacao:
# raw_lower = layer.lower()
# if "vaz" in raw_lower or "void" in raw_lower: return True
```

Referencia completa das classes e funcoes usadas no pipeline RAG para pilars. Todas as classes estao em scripts/ e podem ser importadas diretamente.

PlausibilityChecker (rag_plausibility.py)

Metodo	Parametros	Retorno	Descricao
check()	eid, tipo, entity_data, obra	PlausibilityResult	Consulta RAG top-3
_build_query()	eid, tipo, data	str	Monta query textual para embedding
_classify()	score: float	str	Classifica score em ACEITAR/REVISAR/REJEITAR

StructuralValidator (rag_validator.py)

Metodo	Parametros	Retorno	Descricao
validate()	tipo, eid, data, obra	ValidationResult	Verifica limites dimensionais
_check_limits()	tipo, dim, value	Violation None	Checa 1 dimensao contra limites

AnomalyDetector (rag_anomaly_detector.py)

Metodo	Parametros	Retorno	Descricao
detect()	eid, tipo, data, plaus_result, val_result	AnomalyResult	Score combinado
compute_anomaly_score()	semantic_sim, dim_violations	float	Formula $0.5 * sem + 0.5 * dim$
classify()	score: float	str	NORMAL/INCOMUM/SUSPEITO/ANOMALO

PreStogGate (rag_pre_stog_gate.py)

Metodo	Parametros	Retorno	Descricao
evaluate()	obra, elements	GateResult	Avalia obra inteira contra threshold
_compute_pass_rate()	elements	float	Calcula % de elementos OK

TransformationEngine

Metodo	Parametros	Retorno	Descricao
transform()	element_json	DxfGeometry	Converte JSON em geometria DXF
_resolve_key()	element_json	str	Determina DNA key para lookup

Problema	Causa Provavel	Solucao
Texto P* encontrado mas sem LWPOLYLINE	confidence baixa (< 0.30)	Verificar se layer "Paineis" existe no DXF. Pode estar em layer alternativo. Usar <code>normalize_layer()</code> para checar aliases.
<code>normalize_layer()</code> retorna string vazia	Layer com caracteres nao-ASCII puros	DXF pode ter encoding diferente de CP1252/UTF-8. Tentar <code>chardet.detect()</code> no conteudo raw do layer.
FAISS index nao carrega	Arquivo .faiss ausente ou corrompido	Re-executar <code>scripts/rag_ingestor.py</code> para reconstruir o indice. Verificar se o diretorio <code>data/</code> contem os JSONs de treino.
Confidence sempre 0.50	Somente lajes sinteticas sendo geradas	Nenhum texto <code>RE_LAJE (L*)</code> encontrado no DXF. Verificar se o layer <code>EST-LAJE-TEXT</code> ou <code>NOMENCLATURA</code> contém os IDs.
DXF de saida sem layers	Robot nao criou layers corretamente	Verificar se <code>ezdxf.new("R2010")</code> esta sendo usado (nao R2000 que nao suporta todos os tipos de layer).
Pilar cambotado nao detectado	bulge threshold muito alto	Verificar se o threshold e 0.3 (nao 0.5). Alguns DXFs usam bulge negativos -- usar <code>abs(bulge)</code> na comparacao.
Dimensao extraida invertida ($b > h$)	<code>RE_DIM</code> captura na ordem errada	Sempre aplicar: <code>comp = max(a,b)</code> , <code>larg = min(a,b)</code> . A convencao e comprimento $\geq$ largura.

Termo	Definicao
AutoCAD DXF	Drawing Exchange Format -- formato de intercambio de desenhos AutoCAD.
b (largura)	Menor dimensao da secao transversal de pilar ou viga, em cm.
BA* (balanco)	Viga em balanco -- possui apenas 1 apoio. apoio_fim="" e correto.
bbox	Bounding box -- retangulo envolvente minimo de uma geometria.
bulge	Fator de curvatura em vertices de LWPOLYLINE. bulge > 0.3 indica cambotado.
Cambotado	Pilar com secao nao retangular, com curva na LWPOLYLINE.
Confidence	Score de confianca da extracao (0.0 a 1.0). Usado para threshold de aceitacao.
CP1252	Codificacao de caracteres Windows-1252. Causa problemas em nomes de layers.
DNA key	Chave de identificacao de tipo+subtipo usada pelo TransformationEngine.
ezdxf	Biblioteca Python para leitura e escrita de arquivos DXF.
FAISS	Facebook AI Similarity Search -- indice de vetores para busca por similaridade.
FV (fundo)	Prancha de fundo da viga. Entidade LINE no layer "fundo".
h (altura)	Maior dimensao da secao transversal de viga, ou espessura de laje, em cm.
INSERT	Entidade DXF que representa uma insercao de bloco (ex: garfos, escoras).
LINE	Entidade DXF que representa um segmento de reta (start, end).
LV (lateral)	Paineis laterais da viga. Entidade LWPOLYLINE no layer "Paineis".
LWPOLYLINE	Lightweight Polyline -- entidade DXF com vertices (x,y,bulge).
modelspace (msp)	Espaco de modelo do DXF -- contem todas as entidades.
NBR 6118	Norma brasileira de projeto de estruturas de concreto armado.
NFKD	Normalizacao Unicode (Compatibility Decomposition) -- decompoe caracteres acentuados.
Pe-direito	Altura livre entre pisos. Para pilares, e a altura do pilar.
RAG	Retrieval-Augmented Generation -- enriquecimento de extracao via corpus.
RE_DIM	Regex para capturar dimensoes no formato "NNxMM" ou "NN*MM".
RE_PILAR	Regex para identificar IDs de pilares (P17, PC-1, PILAR 3, etc).
RE_VIGA	Regex para identificar IDs de vigas (V101, BA-5, VB3, etc).
RE_LAJE	Regex para identificar IDs de lajes (L5, Y1, LAJE 1, etc).
Shoelace	Formula para calculo de area de poligono a partir de coordenadas.
STOG	Structural Transformation Output Generator -- motor de geracao DXF.
Sintetica	Laje gerada automaticamente a partir de clusters de textos h=.
TextAssociator	Algoritmo que pareia textos DXF com poligonos por proximidade.

Termo	Definicao
Tramo	Segmento de viga entre dois apoios consecutivos.

v7.0 (2026-03-19) -- Completo

Secao	Descricao
Indice Geral	Tabela de conteudo com 30+ secoes numeradas
Pipeline Overview	Pipeline 7 fases do CAD-ANALYZER com descricao detalhada
Corpus Statistics	Dados reais hardcoded: 228 pilares, 351 vigas, 220 lajes em 11 obras
Cross-Obra	P17 em 7 obras com dimensoes variadas (demonstra unicidade)
RAG Plausibility	Documentacao completa do PlausibilityChecker com codigo
RAG Validator	Documentacao do StructuralValidator com limites NBR 6118
Anomaly Detector	Formula combinada + categorias + exemplos com P_ABS
Pre-STOG Gate	Resultados das 11 obras (todas PASS) + criterios
Robot Integration	Bolt/Crane/Slab robot + DXF output schema
TransformationEngine	DNA key lookup + registry de transformers
CEO-AUDIT D8	Acuracia por tipo de elemento (ground truth)
Casos Especiais	Cambotado, circular, L, balanço, sintetica, vazios
Encoding Guide	normalize_layer(), CP1252 vs UTF-8, tabela de conversoes
API Reference	Todas as classes RAG com metodos e parametros
Troubleshooting	Erros comuns e solucoes por tipo de elemento
Glossario	30+ termos tecnicos definidos

v6.0 (2026-03-19) -- RAG Integration

Secao	Descricao
RAG Section	Top-3 similares do corpus FAISS por elemento
Thresholds	Tabela de decisao ACEITAR/ACEITAR_COM_AVISO/REVISAR/REJEITAR
Anomaly Score	Formula: $0.5 * (1 - \text{sim}) + 0.5 * \text{dim_penalty}$
Integracao Pipeline	Codigo de exemplo para integracao com robot_integration.py

v5.0 (2026-03-19) -- Base Instrutiva

Secao	Descricao
Capa + TOC	Capa por elemento + tabela de conteudo 9-10 secoes
Identificacao	RE_PILAR, RE_VIGA, RE_LAJE com tabelas de exemplos

Secao	Descricao
Associacao	TextAssociator 3 raios + diagrama matplotlib
Secao Transversal	Diagrama pilar retangular + cambotado
Layers	Tabela de layers canonicos + aliases + normalize_layer()
Schema JSON	Schema completo FichaFase3 por tipo de elemento
Confidence	Formula + diagrama visual + cadeia de fallbacks
Matriz Decisao	Todos os casos ambiguos com acao e log
Exemplos Reais	P17, V101, BA-5, L5, synth_0 com DXF e JSON
Pipeline	Fluxo E2E 10-12 passos + diagrama matplotlib

■ Total estimado: v5 = 12-15 paginas por PDF | v6 = +3-5 paginas | v7 = +15-20 paginas. Total v7: 30-40 paginas por PDF.